



Git to GitHub

A Hands-On Foundation for Faculty — Version Control, Demystified

Institution: Vels Institute of Science, Technology & Advanced Studies (VISTAS)

Audience: Faculty — first version control journey

Duration: 2 hours

Level: Basic → Medium fundamentals

Goal: Use Git/GitHub confidently & mentor students for projects, assignments and interview portfolios

CONTENTS

What's Inside This Handout

1	Why This Workshop Matters	3
2	History & Purpose of Version Control	4
3	The Story of Git	6
4	Git Fundamentals — Core Concepts	8
5	Branching & Merging	10
6	Working With Remotes & Good Hygiene	11
7	GitHub Essentials	12
8	Mentoring Students	14
9	Git Command Cheat Sheet	16
10	Key Takeaways & Resources	19

How to Use This Handout

This document mirrors the live workshop slide deck, but in a format you can read at your own pace, annotate, and keep at your desk. Every concept includes a real-world example connecting it to academic work — question papers, lab manuals, research drafts, and student mentoring — so the ideas stick beyond the two-hour session.

Tip: Section 9 (Git Command Cheat Sheet) is designed to be your everyday quick-reference — keep that page bookmarked even after the workshop ends.

SECTION 1

Why This Workshop Matters to You

For Your Teaching

Manage lab manuals, question papers and course material with full history — no more "final_v3_USE_THIS" files. Every revision is timestamped and attributable, useful for NAAC/NBA audit trails too.

For Mentoring

Every internship, hackathon and placement drive today expects students to show a GitHub profile. Companies increasingly screen candidates by their GitHub activity before the first interview round.

For Research

Co-author papers, track reviewer changes, and never lose a version again. Works for LaTeX manuscripts, datasets, and grant proposal drafts shared across institutions.

Real world: Three faculty members co-editing a question bank over email end up with *QB_Final.docx*, *QB_Final_v2.docx*, *QB_Final_Anand_edits.docx* — and nobody knows which one is current, or what changed between them. Version control fixes this permanently.

Our 2 Hours Together

Part	Duration	Covers
Part 1	20 min	History, purpose & types of version control + the story of Git
Part 2	35 min	Git fundamentals — setup, the 3 trees, daily commands, branching & merging
Part 3	15 min	Remotes, undo toolkit & good commit hygiene
Part 4	25 min	GitHub essentials — collaboration, Issues, Projects, Actions, Pages
Part 5	15 min	Mentoring students — workflows, portfolios, do's & don'ts
Wrap-up	10 min	Quick quiz, recap & Q&A

SECTION 2

History & Purpose of Version Control

The Problem Before Version Control

The Familiar Folder Mess

- Syllabus_2023.docx
- Syllabus_2023_revised.docx
- Syllabus_2023_revised_FINAL.docx
- Syllabus_2023_revised_FINAL_HOD_approved.docx
- Syllabus_2023_revised_FINAL_HOD_approved(1).docx

What We Actually Needed

- One file, many **tracked** changes over time
- Know **who** changed **what**, and **why**
- Safely try a new idea without losing the original
- Merge two people's edits automatically
- One single, unambiguous "current" version, always

A Brief History — 50 Years of Version Control

Year	System	What changed
1972	SCCS	Source Code Control System — Bell Labs, first true VCS
1982	RCS	Revision Control System — stores diffs, single-file focus
1990	CVS	Concurrent Versions System — first popular client-server VCS
2000	SVN	Apache Subversion — fixed many CVS flaws, atomic commits
2005	Git & Mercurial	Distributed VCS era begins — speed, branching, no single server
2008+	GitHub era	Social coding — Git gains a collaboration layer in the cloud

Three Types of Version Control Systems

1. Local VCS

A simple database on your own machine tracking file changes over time (e.g. RCS, 1982). Simple, but no sharing or off-machine backup.

2. Centralized VCS

One central server holds the full history; everyone checks out from it (CVS, SVN). Server down ⇒ nobody can commit or see history.

3. Distributed VCS

Every collaborator has a **full copy** of the history (Git, Mercurial). Works offline, no single point of failure.

The Purpose of Version Control

History

Every change is timestamped, attributed and recoverable forever.

Collaboration

Multiple people work on the same project without overwriting each other.

Safe Experimentation

Try a risky change in isolation (a branch) without breaking the working version.

Accountability & Backup

Know exactly who changed what and why — every clone is a full backup.

The Story of Git

Born From a Crisis — April 2005

The Linux kernel — thousands of contributors worldwide — used a proprietary tool called **BitKeeper** to manage its source code for free. In April 2005, the company withdrew the free license after a dispute. Linus Torvalds and the kernel team suddenly had **no version control system** for the world's largest collaborative codebase.

Linus's response: build a new one from scratch. In about **10 days**, the first working version of Git existed.

Design Goals Linus Set

- Must be **fast** (kernel has 1000s of files)
- Fully **distributed** — no single server to lose
- Strong **integrity** guarantees against corruption
- Cheap, instant **branching**
- Support fully non-linear workflows

Fun Fact

"Git" is British slang for an unpleasant person. Linus joked: *"I'm an egotistical bastard, and I name all my projects after myself. First Linux, now Git."*

Why Git, When Others Already Existed?

⚡ Speed

Most operations happen **locally** — no network round-trip. The Linux kernel has 70,000+ files; Git still handles a commit in milliseconds.

🌐 Truly Distributed

Every clone is a complete backup with full history. Commit, branch and review history on a flight with no Wi-Fi; sync once you land.

🌿 Cheap Branching

In CVS/SVN, branching was slow and "scary." In Git it takes milliseconds — so people actually use it.

🔒 Data Integrity

Every snapshot is checksummed (SHA hash). Change one character anywhere in history, and every hash after it changes too.

Git vs. Other Source Code Management Tools

Aspect	CVS / SVN (Centralized)	Git (Distributed)
Repository copy	Only the central server has full history	Every clone has the entire history
Working offline	Very limited	Commit, branch, view history fully offline
Branching	Heavy, slow, often avoided	Instant, lightweight, encouraged
Data model	File-by-file diffs	Whole-project snapshots
Single point of failure	Yes — central server	No — every copy is a full backup

Git vs. GitHub — Not the Same Thing!

Git

A version control **tool**. Runs on your computer. Free & open source. Works with no internet. Like Microsoft Word with Track Changes.

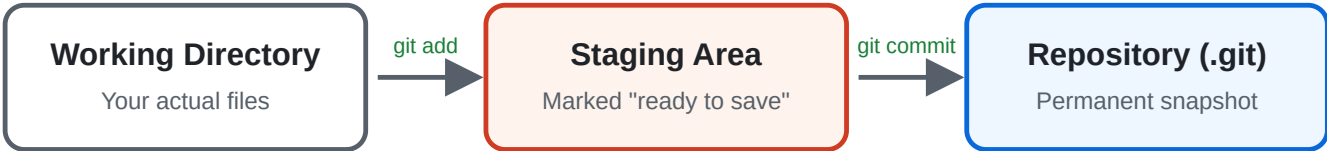
GitHub

A cloud **hosting + collaboration** service. Stores your Git repos online. Adds Issues, PRs, Actions, Pages. Like Google Drive for your Git repos.

Note: GitHub isn't the only such service — GitLab and Bitbucket are popular alternatives that also host Git repositories. Git itself is independent of all of them.

Git Fundamentals — Core Concepts

The Three Areas of Git



Real world: Think of grading assignment scripts. **Working Directory** = the scripts on your desk you're marking. **Staging** = the pile you've finished marking and set aside. **Commit** = officially handing that pile to the exam cell with a signed cover note (the commit message).

Setting Up Git — Tell It Who You Are

`git config --global user.name "Your Name"`

Sets the name attached to every commit on this machine.

`git config --global user.email "you@vistas.ac.in"`

Should match your GitHub account email.

git init vs. git clone

 **git init**

`git init` turns the current folder into a brand-new, empty repository. Use when starting a fresh project from nothing.

 **git clone**

`git clone <url>` downloads a complete copy of an existing repository — files *and* full history.

The Daily Loop — status → add → commit

Command	Purpose
<code>git status</code>	Shows what changed: modified, new (untracked), or staged files. Run it constantly.
<code>git add <file></code>	Stages a change. Use <code>git add .</code> to stage everything changed.
<code>git commit -m "message"</code>	Permanently saves the staged snapshot to history with a description.

Looking Back — log, diff, show

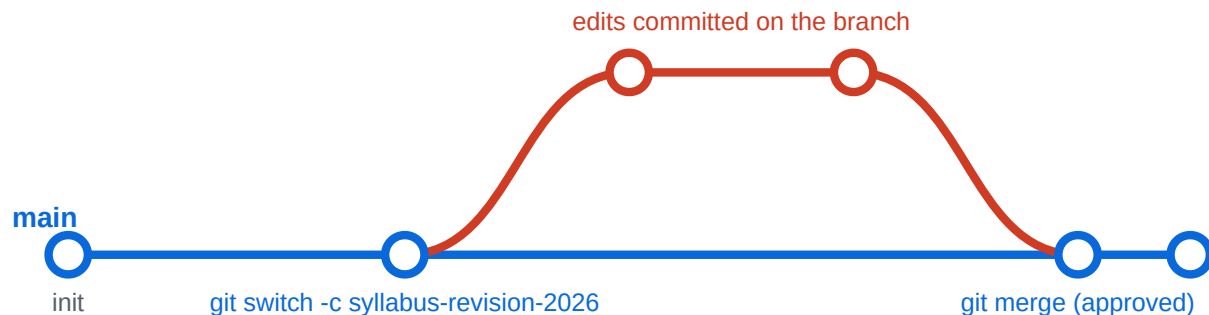
Command	Purpose
<code>git log --oneline --graph</code>	A compact, visual view of history — great for seeing branches.
<code>git diff</code>	Shows exact line-by-line changes not yet staged.
<code>git show <commit></code>	Shows full details and changes of one specific commit.
<code>git blame <file></code>	Shows who last changed each line, and in which commit.

Real world: A student's marks look wrong in the results sheet. `git blame` shows exactly who edited that row, and `git show` reveals exactly what changed and why — a complete audit trail.

SECTION 5

Branching & Merging

Why Branches Matter



A branch is an independent, movable line of development — a parallel copy of the project where you can experiment freely. The **main** branch is the stable, "always working" version. Create a new branch to build a feature or try an idea, then **merge** it back once ready.

Real world: You're revising a syllabus and don't want to risk the version currently in use by students. The `main` branch stays the approved syllabus; `syllabus-revision-2026` holds your draft. Once the Board of Studies approves it, you merge into `main`.

Branch Commands

Command	Purpose
<code>git branch</code>	List all local branches; current one is marked with *.
<code>git switch -c <name></code>	Create AND switch to a new branch in one step.
<code>git merge <name></code>	Merge the named branch's changes into your current branch.
<code>git branch -d <name></code>	Delete a branch once it's been merged in safely.

Merge Conflicts — What & How

Two branches changed the **same lines** of the **same file** differently. Git can't decide which version is "right" — it asks you.

```
questionpaper.md
<<<<<<< HEAD (main)
Q5. Explain normalization in DBMS. (5 marks)
=====
Q5. Explain database normalization with an example. (8 marks)
>>>>>> co-faculty-edits
```

To resolve: open the file, read both versions, manually edit to keep the correct/combined content, remove the conflict markers, then `git add <file>` and `git commit`.

Real world: You and a co-faculty member both edited Question 5 of the same question paper in your own branches — one rewords it, the other changes the marks. Git flags this exact line for a human decision.

Working With Remotes & Good Hygiene

Syncing With a Remote Repository

Command	Purpose
<code>git remote add origin <url></code>	Link your local repo to a remote one, named "origin".
<code>git remote -v</code>	View the remotes currently configured.
<code>git push origin main</code>	Upload your local commits to the remote branch.
<code>git pull origin main</code>	Download & merge remote changes into your local branch.
<code>git fetch</code>	Download remote changes WITHOUT merging — inspect first.

The "Oops" / Undo Toolkit

Situation	Command	Effect
Undo edits, not yet staged	<code>git restore <file></code>	Reverts file to last committed version
Undo last commit, keep changes	<code>git reset --soft HEAD~1</code>	Commit removed, changes stay staged
Undo a commit already pushed	<code>git revert <commit></code>	✓ Safe — adds a new commit that undoes it
Shelve unfinished work	<code>git stash</code>	Temporarily shelves changes; <code>git stash pop</code> restores

Golden rule: Once a commit is pushed and others may have it, prefer `git revert` over `git reset --hard` — rewriting shared history causes chaos for collaborators.

.gitignore & Good Commit Messages

.gitignore

A file listing things Git should **never** track — temp files, passwords, build outputs.

```
*.log
node_modules/
.env
```

Commit Messages

~~"fixed stuff", "update", "final final"~~

Good: "Fix mark calculation bug in CO2 evaluation sheet"

GitHub Essentials

From Local Folder to GitHub Repository

Option A: Start on GitHub

- Click "New repository" on GitHub
- Name it, add a README
- `git clone <url>` on your machine
- Add files, commit, push

Option B: Start Locally

- `git init` in your project folder
- Create the empty repo on GitHub
- `git remote add origin <url>`
- `git push -u origin main`

Fork vs. Clone, and the Pull Request

Fork vs. Clone

Clone = download a copy to your machine. **Fork** = your own copy of someone else's repo *on GitHub itself* — you fully own it and can propose changes back.

Pull Request (PR)

A formal request: "Please review and merge my branch's changes." GitHub's core collaboration mechanism — code review happens here, line by line.

Issues, Labels & Project Boards



Issues

Track bugs, tasks, or ideas as individual trackable items with discussion threads.



Labels & Milestones

Categorize ("bug", "enhancement") and group issues into deadlines/releases.



Projects (Kanban)

Drag-and-drop boards — "To Do / In Progress / Done" — built into the repo.

Real world: Coordinating a final-year project team of 4 students? Create an Issue for each module, assign owners, and watch the Project board move from To Do to Done — exactly how industry teams manage software work.

Automation & Polish



GitHub Actions

Automate tasks on every push — run tests, check code style, even auto-grade assignments.



GitHub Pages

Host a free website directly from a repository — perfect for a portfolio or demo page.



Wiki & README

Built-in documentation space — explain setup, usage and design decisions.

SECTION 8

Mentoring Students

Standard Workflow for Assignments & Projects

Fork class repo → Clone locally → `git switch -c assignment-1` → Code + commit often → `git push`
→ Open PR for submission → Faculty reviews via comments

Why this is powerful for grading: The Pull Request becomes the submission. You can comment directly on specific lines of code, see exact commit timestamps (useful for late-submission policies), and students build real industry habits from day one.

Building a Campus-Interview-Ready Portfolio

What Recruiters Look At

- 📌 Pinned repositories — their 6 best projects
- 📊 Contribution graph — consistency over time
- 📄 Quality README per project
- 🌱 A profile README (`username/username` repo)

Guidance to Give Students

- Commit regularly (not one giant commit the night before)
- Write descriptive commit messages
- Always add a README with setup instructions
- Showcase 3-4 polished projects rather than 30 empty ones

Do's & Don'ts to Teach Students

✅ Do

- ✓ Commit small, logical chunks of work
- ✓ Pull before you start working each session
- ✓ Use branches for every new feature/assignment part
- ✓ Write a README for every project

❌ Don't

- ✗ Commit passwords, API keys, or `.env` files
- ✗ Use `git push --force` on a shared/team branch
- ✗ Make one giant commit with "final project" as the message
- ✗ Panic and delete the repo when a conflict appears

SECTION 9

Git Command Cheat Sheet

First-Time Setup

Command	Purpose
<code>git config --global user.name "Your Name"</code>	Set the name attached to your commits
<code>git config --global user.email "you@vistas.ac.in"</code>	Set the email attached to your commits
<code>git config --list</code>	View all current settings

Starting a Project

Command	Purpose
<code>git init</code>	Turn the current folder into a new, empty repository
<code>git clone <repo-url></code>	Download a full copy (files + history) of an existing repository

The Daily Loop

Command	Purpose
<code>git status</code>	Show what's changed, staged, or untracked
<code>git add .</code>	Stage all changed files
<code>git commit -m "message"</code>	Save the staged snapshot permanently
<code>git commit --amend</code>	Edit the most recent commit

Looking at History

Command	Purpose
<code>git log --oneline --graph --all</code>	Visual graph of all branches
<code>git diff</code>	Line-by-line changes not yet staged
<code>git diff --staged</code>	Line-by-line changes that ARE staged
<code>git show <commit-hash></code>	Full details of one specific commit
<code>git blame <file></code>	Who last changed each line, and in which commit

Git Command Cheat Sheet

Branching & Merging

Command	Purpose
<code>git branch</code>	List local branches (current marked with *)
<code>git switch -c <name></code>	Create and switch to a new branch
<code>git merge <name></code>	Merge a branch into your current branch
<code>git branch -d <name></code>	Delete a branch (safe — only if merged)
<code>git branch -a</code>	List all branches, including remote-tracking ones

Working With Remotes

Command	Purpose
<code>git remote add origin <url></code>	Link your local repo to a remote, named "origin"
<code>git remote -v</code>	View configured remotes
<code>git push -u origin main</code>	Push and remember this link for future push/pull
<code>git pull origin main</code>	Download and merge remote changes
<code>git fetch</code>	Download remote changes without merging

The "Oops" / Undo Toolkit

Situation	Command
Undo edits, not yet staged	<code>git restore <file></code>
Accidentally staged a file	<code>git restore --staged <file></code>
Undo last commit, keep changes staged	<code>git reset --soft HEAD~1</code>
Undo a commit already pushed/shared	<code>git revert <commit></code>
Shelve unfinished work temporarily	<code>git stash</code> / <code>git stash pop</code>

Ignoring Files (.gitignore)

```

*.log
node_modules/
.env
__pycache__/
*.pyc
.DS_Store

```

GitHub-Specific Concepts

Term	Meaning
Fork	Your own copy of someone else's repository, on GitHub
Pull Request (PR)	A formal request to merge your branch's changes, with line-by-line review
Issue	A trackable task, bug report, or idea with discussion
GitHub Actions	Automation that runs on events (e.g. test code on every push)
GitHub Pages	Free static website hosting directly from a repository

Quick Troubleshooting

Problem	Likely Fix
"fatal: not a git repository"	Run <code>git init</code> or <code>cd</code> into the right folder
Merge conflict markers in a file	Resolve manually, then <code>git add</code> + <code>git commit</code>
Accidentally committed a secret	Add to <code>.gitignore</code> , <code>git rm --cached <file></code> , rotate the credential
"Your branch is behind 'origin/main'"	<code>git pull origin main</code> before pushing again

Key Takeaways & Resources

- ✓ Version control solves the "which file is the real final version" problem forever
- ✓ Git is **distributed** — every clone is a complete, independent backup
- ✓ Git ≠ GitHub: tool vs. cloud collaboration platform
- ✓ The daily loop is `status → add → commit`; sync with `push/pull`
- ✓ Branches make experimentation safe and cheap — use them freely
- ✓ GitHub's Issues/Projects/Actions turn version control into full project management
- ✓ Guiding students through Fork → Branch → Commit → PR builds real industry habits and interview-ready portfolios

Resources From This Workshop



Live Slide Deck

`index.html` — the interactive keyboard-navigable presentation used in the live session, including an 8-question quiz.



Command Cheat Sheet

`cheatsheet.md` — the same quick-reference as Section 9 of this handout, in Markdown for easy searching.

Practice is everything. Try the daily loop (`status → add → commit`) on one real document this week — a syllabus, a question paper draft, or a set of lecture notes. The habit forms faster than the theory.